

PORYZ — the value function, rebuilt around positions instead of totals

Spec, 2026-07-28. Nothing here is implemented yet. Every number quoted comes from a run.

Why not reform PORY2

Reform implies the design is sound and the execution is off. Three measurements say otherwise.

Its edge over counting bodies is 3.4 points. 63.6% against a 60.2% material-sign baseline, on held-out human positions. Fourteen of its seventeen features are collectively buying almost nothing over *who has more Pokémon left*.

More data makes it slightly worse. Eight times the training set moved accuracy -0.2 points. That is not undertraining. That is a model at its ceiling.

It cannot see the position that beat us. Will's Bellibolt turn: his partner at full HP, about to be at 10%, about to die to its own spread move. PORY2's inputs are `hp_total_diff` and `alive_diff`. Feed it that board and it reports a healthy position. The thing that mattered was not in the representation and cannot be added by retuning.

So: new name, new lineage entry. PORY and PORY2 stay separately quotable — this repository once had a file claiming 61,274 games from a store holding 20,387, and the rule that came out of it is that two versions never share a name.

The one structural change

Per-Pokémon, not per-side.

Every current feature is a difference of aggregates. `hp_total_diff` says "I have 200 more HP than you". That is the same number whether it is spread across two healthy Pokémon or sitting on one healthy body next to one at 4%. Those are completely different positions and VGC is almost entirely about which.

Aggregates cannot express *this specific thing is in range*, and being in range is what a position mostly consists of.

The representation

For each of the (up to) four active Pokémon and the benched remainder:

- HP fraction, status, every stat stage
- item and ability *as revealed by the open team sheet* — which the player now reads (2026-07-28)
- typing, and the four moves when the sheet publishes them
- speed, computed the way `board.js` already computes it: investment-aware, nature-narrowed, Choice Scarf, paralysis, Tailwind, Trick Room

And then the part that is currently missing entirely:

- **the threat matrix.** For each attacker/defender pair across the field: expected damage fraction, kill probability, and who moves first.

MAG already computes every one of those, per turn, for its own decisions. PORY2 recomputes a cruder version from scratch and throws MAG's away. PORYZ should call the same code — one damage engine, one speed order, one definition of "in range".

Why it can be a network, when PORY2's could not

PORY2's header records that both neural variants **lost** to a logistic model. That result stands and it is not an argument against a network here — it is an argument about what it was fed.

Seventeen features that are mostly `my total minus your total` have no interactions to find. A linear model extracts essentially everything available and a network has nothing left to add.

The representation above is hundreds of inputs whose entire content *is* interaction: this mon threatens that mon, but only while it is faster, but Trick Room is up, but it is itself in range of a spread move. A linear model cannot express that and a network can.

So the order matters: fix the representation first, then the network is justified rather than fashionable. Reaching for a net today would lose again and teach nothing.

What it trains on

Open-sheet self-play generated by the win-optimised policy. Both halves of that are corrections found today.

Open sheet, because self-play had never played one — zero `|showteam|` events across 1,934 games, and structurally impossible in the ladder format since the rule resolves through a chat command that does not exist in the simulator stream. Every position PORY2 ever trained on came from a game where neither side could see the other's sheet. That is not the format.

Win-optimised policy, because PORY2 learns from games MAG played against itself while MAG was fitted to imitate humans. More games of two mediocre imitators does not teach what a good position looks like. **PORY2 is not data-starved, it is policy-starved**, and that is the honest reading of a flat learning curve.

Validation stays on held-out **human** games, never trained on. That is the only detector for self-play drifting into its own habits, and dropping it would make the whole thing unfalsifiable.

What it is for

The reason to have a value function at all: MAG currently has no concept of *and then what*. It scores this turn's moves and clicks. Every mistake Will found that was not a missing feature was this — including the Bellibolt turn, which no feature can fix because the ally was healthy at decision time and the lethal state only existed *after* both sides moved.

PORYZ is the leaf of:

$EV(a) = \text{SUM over their likely replies } P(\text{reply}) \times V(\text{board after})$

`P(reply)` is already unblocked: MAG's move distribution is calibrated well enough to sample from (ECE 0.0041 held out), which was measured specifically to answer this.

Engineering constraints, stated up front

It has to be callable from Node. MAG decides in JavaScript. PORY2 exists only inside a Python process that prints accuracy and exits — `data/porygon2.json` is 1.7 KB of results with no model in it, no training matrix, no normalisation constants. Nothing downstream can consume it. That is the "clean outputs that flow into dependent models" rule failing at the first hop, and PORYZ must export a loadable artifact or it will be the fourth model that is fitted and never wired in.

It has to be fast enough for the EV sum. One leaf evaluation per (my move $\times$ their likely reply) is roughly 50 per turn. Tolerable live; far too slow for the volumes MACHAMP needs if each one is a scan over 73,000 rows. Either a compact fitted model or a spatial index — decide with a measurement, not a preference.

Gates

Nothing ships because it should work. Today alone: DODUO had the better argument and lost, the mechanics batch had the better argument and did nothing, and the forfeit theory was mine and was wrong.

1. **Beats material sign by more than 3.4 points** on held-out human positions. Otherwise it is PORY2 with more code.
2. **The learning curve is not flat.** If eight times the data still does nothing, the problem was never the policy and this spec's central claim is wrong.
3. **It separates the Bellibolt class of position** — partner healthy now, lethal after the opponent moves — from a genuinely healthy one. If it cannot, it does not solve the thing it was built for.
4. **The bot using it beats the bot without it**, paired, head to head. Prediction is not play.
5. **It passes** `tests/test-wiring.js` — it must prove it ran inside a real game, because three models in this repository were fitted, saved, quoted and never once in a live decision.